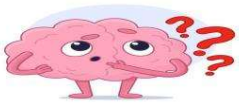
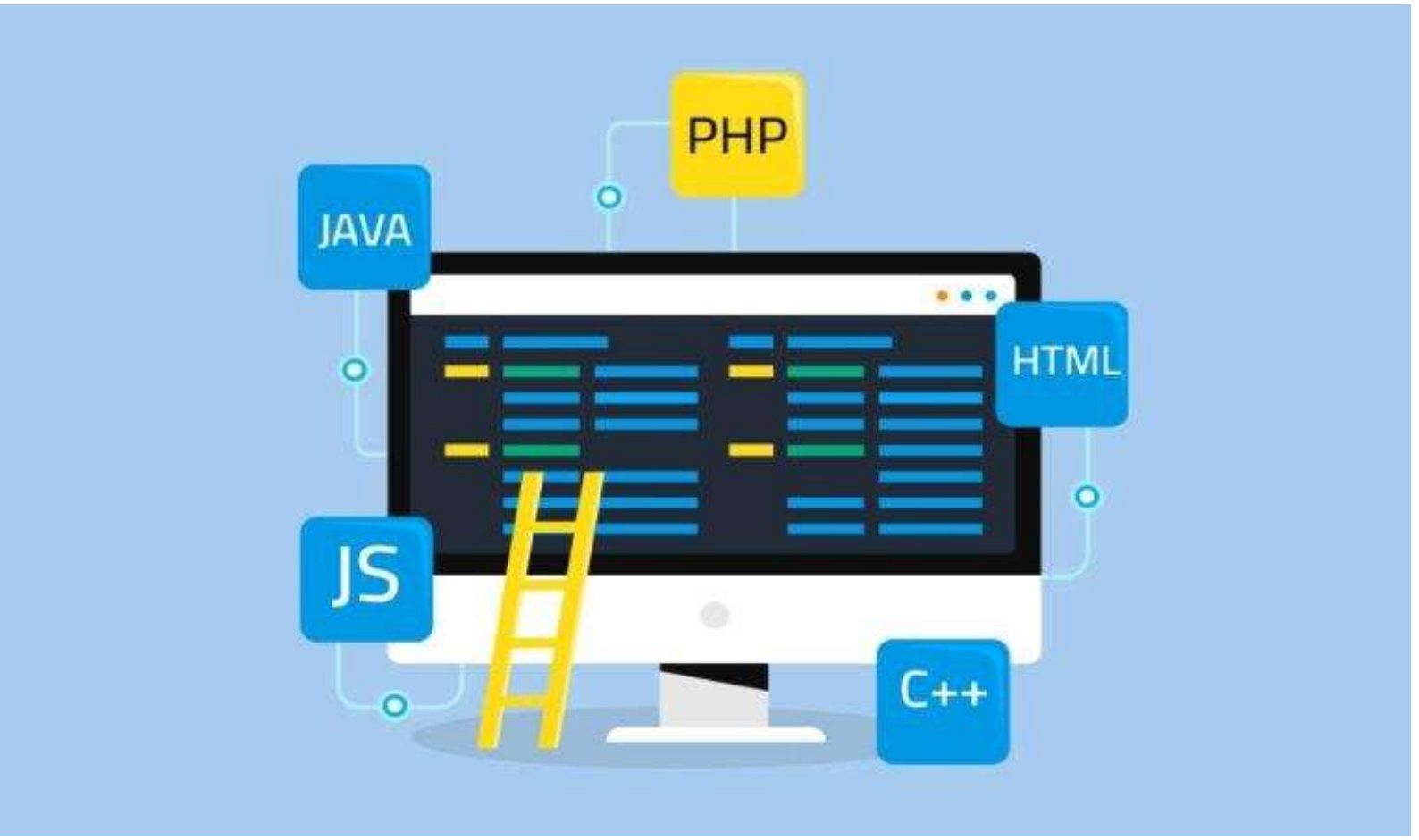
# قبل أن يُكتب الكود



(من الفكرة إلى التخطيط)




- اسم الطالبة: مَيّ تامر إسماعيل السعيد
- الرقم الجامعي: 250281
- اسم المُقرر: تطبيقات الإنترنت
- عنوان البحث: أهمية التخطيط ورسم هيكل الموقع قبل البرمجة

المُقدمة:

هل التخطيط ورسم الهيكل خطوة إضافية أو رفاهية؟



التخطيط ورسم هيكل الموقع قبل البرمجة هو الخطوة الأساسية والعمود الفقري لنجاح أي مشروع رقمي!

حيثُ أنك عندما تقرر البدء في كتابة الأكواد البرمجية بشكل مُباشر دون تخطيط وطريق واضح، يُشبه بناء منزل دون مخطط هندسي.



نتيجة ذلك؟ ☹

- ظهور أخطاء هيكلية قد تُودي بهلاك المشروع.
- عشوائية التصميم والواجهات.
- سلسلة لا نهائية من التعديلات.
- تجربة مُستخدم [زائر] سيئة.
- هدر الوقت والمال.

☺ **إذًا ما فائدة رسم هيكل الموقع؟**

- **خريطة طريق مرئية، توزيع العناصر بذكاء، سهولة التعديل المبكر.**

**الاستثمار في أسبوعين من التخطيط والرسم يُوفر شهورًا من التعديلات البرمجية المُكلفة لاحقًا! ➢**

➢ بدايةً، ما الفرق بين مُخطط الموقع ورسم هيكل الموقع؟

❑ من حيث:- [التركيز، الشكل، الغرض]

| ★ وجه المُقارنة | ★ مُخطط الموقع (Sitemap) | ★ رسم هيكل الموقع (Wireframe) |
|---|---|---|
| ➢ التركيز | → يُركز على:<br>▪ عرض الهيكل العام للموقع،<br>▪ وكيفية تنقل المُستخدمين بين الصفحات. | → يُركز على:<br>▪ توزيع عناصر الصفحة ومواقعها، ك: [القوائم، النصوص، الصور، الأزرار] |
| ➢ الشكل | ▪ مُخطط شجري أو قائمة بأسماء الصفحات. | ▪ رسم تخطيطي،<br>▪ غالبًا ما يكونان باللونين الأبيض والأسود. |
| ➢ الغرض | ▪ تنظيم بنية المعلومات،<br>▪ ومُساعدة مُحركات البحث على أرشفة الموقع. | ▪ تحديد تجربة المُستخدم (UX) ومنطق التفاعل قبل الانتقال لتصميم واجهات المظهر (UI) أو البدء بالبرمجة. |

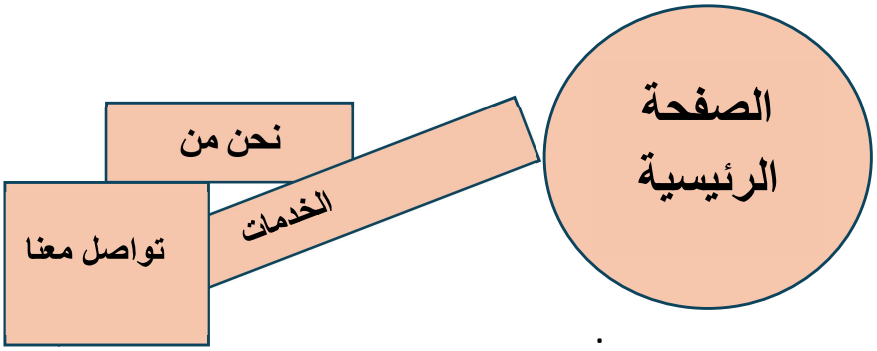
❑ من حيث:- [النطاق، الشبه، زمن الاستخدام]

| ★ وجه المُقارنة | ★ مُخطط الموقع (Sitemap) | ★ رسم هيكل الموقع (Wireframe) |
|---|---|---|
| ➢ النطاق | ▪ الموقع بالكامل (كل الصفحات وعلاقاتها) | ▪ صفحة واحدة أو شاشة مُعينة. |
| ➢ ماذا يُشبه في الواقع؟ | ▪ فهرس أو جدول محتويات كتاب. | ▪ المسودة الأولى لتصميم ما. |
| ➢ متى يُستخدم؟ | ▪ في المرحلة الباكرة للتخطيط. | ▪ بعد تحديد الصفحات وقبل التصميم والبدء بالبرمجة. |

❑ مثال توضيحي على مُخطط الموقع (Sitemap):

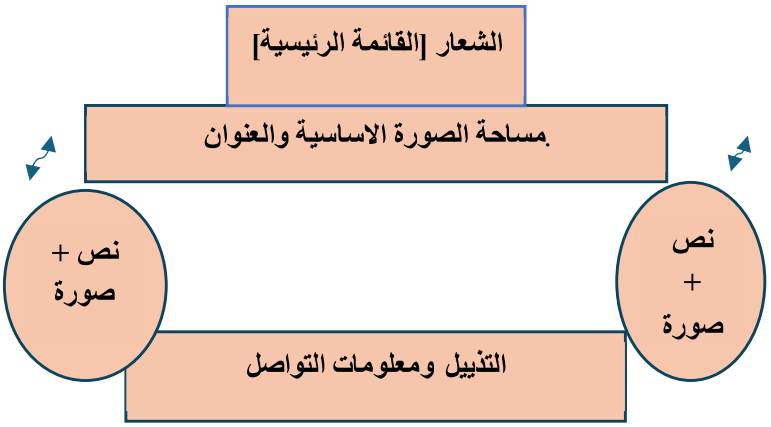
مخطط الموقع يوضح الصفحة الرئيسية، وعلاقتها بالصفحات الفرعية، وكيفية تنظيمها وتسلسلها الهرمي، ويُجيب على سُؤال:

(ما صفحات الموقع وكيف ترتبط ببعضها؟)



❑ مثال توضيحي على رسم هيكل الموقع (Wireframe):

رسم هيكل الموقع يُوضح أماكن العنصر داخل الصفحة، وليس مُجرد أسماء الصفحات، ويُجيب على سُؤال: (أين ستُوضع العناصر داخل كل صفحة؟)




➢ أهمية التخطيط ورسم هيكل الموقع قبل البرمجة؟

1. تحديد تجربة المُستخدم (UX) وواجهة المُستخدم (UI):

[ يُساعد رسم الهيكل في دراسة رحلة المُستخدم داخل الموقع بدقة، وضمان سهولة الوصول إلى الصفحات والأزرار، ونتيجة ذلك تكمُن في اكتساب رضا وقُبول الزوار.]

2. تحسين البنية البرمجية وقواعد البيانات:

[يُساعد التخطيط في فهم كيفية تدفق البيانات بين الصفحات، وتحديد المُكونات التي يُمكن إعادة استخدامها، مما يسهل كتابة كود بشكل احترافي وبشكل مُنظم.]

3. سُهولة التواصل بين فريق العمل:

[يعمل الهيكل البصري كلُغة مُشتركة تجمع بين: المُصممين والمُبرمجين والعُملاء؛ لضمان توافق الجميع على رُؤية واحدة دُون حُدوث خلل أو سُوء تفاهم.]

4. توفير الوقت والتكلفة الفعالة: [التعديل على تصميم برمجي أو هيكل الموقع يكُون أيسر وأقل تكلفة

من التعديل على الأكواد البرمجية، كـ: {HTML,CSS,JAVA SKRIPT}]

➢ دور المُخططات البصرية باستخدام أدوات مثل: draw.io في تنظيم وتخيل المنصة؟

آداة DRAW.IO:

- تُصنف كآداة مجانية لإنشاء المُخططات الهيكلية (WIREFRAMES) ومُخططات تدفق البيانات ((FLOWCHARTS
- تلعب دورًا مُهمًا في المرحلة التي تسبق البرمجة من خلال:
-

1. تخيل وتجسيد شكل المنصة البصري (VISUALIZATION)

✓ [تُتيح الآداة سحب وإسقاط عناصر جاهزة تُمثل مكونات صفحة الويب كـ: الأزرار، الحقول، الصور، النصوص:، وهذا يُساعد المُطور على رُؤية تخيل مبدئي لشاشات الموقع، ومكان كل عنصر قبل كتابة وُسوم الـ HTML، مثل: <header> , <nav> , 

2. رسم مُخطط تدفق البيانات وسير العمل (USER FLOW CHART):

✓ [يُمكن استخدام الآداة لرسم مُخططات تُوضح ماذا يحدُث عندما يضغط المُستخدم على زر مُعين، مثال: إذا ضغطنا على "تسجيل الدخول" -> هل ستفتح نافذة مُنبثقة أم سينتقل إلى صفحة جديدة؟

3. تصميم هيكل قواعد البيانات (ER DIAGRAM):

✓ [تُساعد الآداة في رسم المُخططات التي تُوضح الجداول والعلاقات بين البيانات كـ: علاقة المُستخدم بالمُنتجات أو الطلبات:، وذاك قبل كتابة الأكواد الخلفية -BACKEND-]

4. التنظيم وسُهولة التعديل والتكامل:

✓ [تدعم الآداة حفظ الملفات السحابية، ومُشاركتها مع فريق العمل، مما يسمح بالتعديل الفوري بناءً على المُلاحظات، قبل اعتماد التصميم النهائي للبرمجة.]

❑ مُقارنة بين العمل بمُخطط بصري والبرمجة المُباشرة؟

| ➢ وجه المُقارنة | ➢ البرمجة المُباشرة -دون تخطيط- | ➢ الاعتماد على التخطيط البصري (DRAW.IO) |
|---|---|---|
| 1. نسبة الخطأ | مُرتفعة بسبب تشتت الأفكار أثناء كتابة الكود. | مُنخفضة جدًا لوجود مرجع بصري واضح. |

| | | |
|---|---|---|
| 2. تعديل التصميم | مُعقد ويستهلك وقتًا لإعادة كتابة كود الـ: HTML /CSS | سهل وفوري عبر السحب والإسقاط. |
| 3. فهم العميل للمشروع | صعب حتى يرى المُنتج النهائي. | سهل جدًا من المراحل الأولى. |

➢ المراجع:


1. Garrett, J. J. (2010). The Elements of User Experience: User-Centered Design for the Web and Beyond. New Riders.
(كتاب أساسي يشرح أهمية المعمارية المعلوماتية ورسم الهياكل للمواقع).

2. Interaction Design Foundation. What is Wireframing?
(دليل شامل حول أهمية التخطيط الهيكلي في تحسين تجربة المستخدم قبل البرمجة).

3. draw.io Official Documentation (diagrams.net). Flowchart Maker and Online Diagram Software.
(يوضح استخدامات الأداة في تصميم هياكل المواقع وقواعد البيانات وقنوات التدفق).

4. Duckett, J. (2011). HTML and CSS: Design and Build Websites. Wiley.
يشرح في فصوله الأخيرة كيفية التخطيط للموقع ورسم الهيكل السلكي قبل البدء بكتابة وسوم HTML).


➢